

26-04-14 펌웨어 제작 1일차

26-04-14 펌웨어 제작 1일차

펌웨어는 한 모듈(또는 한 계층)씩 끝까지 검증한 뒤 다음 단계로 넘어간다. 각 단계는 “코드가 있다”가 아니라 “정해진 테스트로 통과했다”를 완료 조건으로 둔다.

정본 문서:

- 26-04-11 session 기반 동작 총정리본.md — 논리 필드·type·module_type·ACK 계약
- 26-04-11 리비타 링크 펌웨어 정리.md — Zephyr·Device Manager·모듈·큐·LoRA·Power

전체 로드맵(요약)

순서	단계	한 줄 완료 조건
1	GitHub 공유	클론·빌드·문서 위치가 README에 고정됨
2	보드·Zephyr 설정	동일 커밋으로 재현 가능한 빌드·로그 출력
3	프로토콜 직렬화	바이트열 인코드·디코드가 표·fixture와 일치
4	LoRA 모듈	하향 수신→ACK 또는 상향 송신이 계약대로 동작
5	Device Manager	DEACTIVATED → ACTIVATING → ACTIVATED_NORMAL 탈출·전이·모듈 set_state·CRON 진입점이 검증됨
6	Power 모듈	배터리 mV·CRON 슬롯이 스펙·로그로 확인
7	통합 검증	Power DATA 가 LoRA 경로로 올라가 상대(스니퍼/목 타워)에서 검증

LoRA(4)까지는 생명주기 없이 main 등 얇은 부트스트랩으로 라디오·직렬화·큐만 검증하고, 그다음(5)에서 Device Manager와 딥슬립 탈출을 한꺼번에 엮어 회귀한다.

진행 방식(공통)

- 단계마다 Git 브랜치 또는 태그로 스냅샷을 남긴다. 예: milestone/02-board , milestone/04-lora , milestone/05-dm-lifecycle .
- 테스트는 가능한 한 자동화한다. 불가능하면 체크리스트 + 캡처·로그 파일을 저장소 test/logs/ 또는 이슈에 첨부한다.
- session 문서의 JSON은 필드 의미 설명용이다. 전선(LoRa 등)에서는 짧은 바이트 직렬화가 정본이므로, 3 단계에서는 “JSON과 동일 필드 의미를 갖는 바이트 레이아웃”을 구현하고, PC 쪽 검증용으로만 JSON·hex dump를 병행해도 된다.
- 다음 단계로 넘어가기 전에 현재 단계의 통과 기준을 전부 만족하는지 리뷰한다.

1. GitHub 공유

목적

협업·CI·버전 고정. 이후 모든 단계의 기준점이 된다.

구성 권장

- `README.md` : 레포 목적, Zephyr 버전, 워크스페이스 초기화 명령, 빌드 명령, 플래시 방법
- `doc/` : Obsidian 정본을 복사하지 않을 경우, 최소한 "정본 경로·날짜·커밋 해시를 맞출 것"을 적은 `PROTOCOL.md` 또는 링크 모음
- `.gitignore` : 빌드 산출물, 로컬 `west` 관련(정책에 맞게)
- 선택: GitHub Actions로 `west build` 만이라도 실행

테스트(완료 조건)

- 다른 PC에서 클론 후 README대로 빌드가 한 번 성공한다(또는 CI가 초록).
- "정본 문서는 어디를 보라"가 README 한 곳에 적혀 있다.

2. 보드·Zephyr 설정

목적

실기판에서 커널이 뜨고, 이후 모든 디버깅의 전제(시리얼·시각·핀)가 잡힌다.

산출물

- 보드별 `board.cmake` / `devicetree` / `prj.conf` 가 레포에 포함
- `main` 또는 최소 앱에서 부팅 로그 1회 이상

테스트(완료 조건)

- 전원 리셋 후 매번 동일한 부팅 메시지(버전·보드명)가 시리얼에 출력된다.
- RTC 또는 시스템 틱 중, 문서에서 요구하는 시각 기준이 될 소스가 하나 확보됐다(정확도는 이후 단계에서 다듬어도 됨).

3. JSON(문서) 기준 프로토콜 — 바이트 직렬화

목적

`session` 문서의 필드 정의와 동일한 의미를 갖는 `encode` / `decode` 를 C(또는 선택한 언어)로 고정한다. 타입별 본문·공통 헤더·ACK 무본문 등은 26-04-11 `session` 기반 동작 총정리본.md §1·§8과 펌웨어 정리의 바이트 표 후보(§6.4 등)를 단일 표로 맞춘다.

산출물

- 공통 헤더 + 최소 1종 상향(`DATA` 등)·ACK 에 대한 인코더·디코더
- 필드 순서·엔디안·고정 길이 패딩 규칙이 코드 주석 또는 `doc/` 에 문서화됨
- 호스트에서 돌아가는 테스트 입력: hex 또는 바이너리 fixture 파일

테스트(완료 조건)

- 표에 적은 예시 패킷 N개에 대해 round-trip(`encode` → `decode`)이 필드 단위로 일치한다.
- ACK 는 본문 길이 0(또는 고정 레이아웃에서 본문 슬롯 없음)이 깨지지 않는다.
- 잘못된 길이·잘못된 `type` 에 대해 실패가 명시적으로 반환된다(펌웨어에서는 `false` / error code 등).

이 단계에서는 RF 없이 순수 라이브러리 테스트만으로 완료 가능하다.

4. LoRA 모듈 작성

목적

수신 큐·송신 큐·need_ack·in-flight·재전송 정책의 뼈대를 session ACK 계약과 맞춘다. 펌웨어 정리 §6·§2.3.

부트스트랩(이 단계 한정)

Device Manager 생명주기는 아직 넣지 않는다. main (또는 임시 보드 초기화 코드)에서 GPIO/SPI·LoRA 드라이버·직렬화 API만 호출해 RF·큐를 검증한다. 펌웨어 정리 §0의 “타워 없이 개발” 분기가 있으면, 그때 쓰는 테스트 전용 진입을 여기서 명시해 두고 문서 정보와의 차이를 주석으로 남긴다.

산출물

- lora_rx → 디코드 → (필요 시) ACK 송신 또는 모듈 큐 라우팅
- lora_module_enqueue_tx 로 상향 패킷 enqueue
- RF 실패·타임아웃 시 로그 또는 카운터

테스트(완료 조건)

- 테스트 A(하드 없음 가능): 3단계에서 만든 바이트 패킷을 RX 경로에 주입했을 때 ACK 가 생성되거나 상향 타입이 올바르게 분류된다.
- 테스트 B(RF): 타워 또는 스니퍼와 1패킷 왕복, packet_id 보존이 로그로 보인다.
- 큐 적재 실패 시 펌웨어 정리 §2.3.2 에 맞는 상향(또는 스텝이라도 한 경로)이 남도록 할지 이 단계에서 결정하고, 최소 한 번 시뮬레이션한다.

5. Device Manager 작성(생명주기 포함)

목적

- DEACTIVATED 에서 ACTIVATING 을 거쳐 ACTIVATED_NORMAL 등으로 넘어가는 탈출·전이(딥슬립·비RTOS 경로·버튼 GPIO). 펌웨어 정리 §3.6, §3.7, 버튼 §4.
- 전이에 맞춰 각 모듈 xxx_module_set_state() 호출, CRON 유틸·스케줄 정책 진입점. 펌웨어 정리 §3·§2.7.

산출물

- 상태 enum, DEACTIVATED → ACTIVATING → ACTIVATED_NORMAL (및 문서상 그 다음 허용 전이) 테이블
- 전이 시 로그(문자열 또는 숫자 코드)
- 모듈별 MODULE_STATE_ACTIVE / INACTIVE 전환 혹(내부는 스텝 가능하나 LoRA는 4단계 동작을 유지하도록 연결)
- CRON 유틸 API(실제 k_timer 무장 주체는 모듈이어도 됨)

테스트(완료 조건)

- 냉부팅 직후 기대 초기 상태(DEACTIVATED 등).
- 탈출 트리거(버튼·주입) 한 번에 ACTIVATING → ACTIVATED_NORMAL 순서가 시리얼(또는 RTT)에 재현된다.
- 탈출 없이 장시간 대기 시 의도치 않은 전이가 없다(최소 스모크).

- LoRA: 4단계에서 통과한 RX 주입·RF 왕복을, 5단계 통합 빌드에서 다시 한 번 돌려 회귀한다(전이 후 `MODULE_STATE_ACTIVE` 에서 동일 동작).

6. Power 모듈 작성

목적

배터리 ADC·mV 환산·(문서대로) CRON 슬롯에서 읽기, `MODULE_STATE` 와 저전압 플래그. 펌웨어 정리 §5 .

산출물

- `power_module_get_battery_mv()` 또는 동등 API
- CRON 마스크·슬롯 만료 시 콜백에서 측정하는 경로
- 선택: `NOTIFY` 전이 조건(저전압·복구)은 이 단계 또는 7단계에서 범위를 정함

테스트(완료 조건)

- 알려진 전압(가변 전원 또는 분압)에서 mV가 오차 범위 내이다.
- CRON을 짧은 간격으로 줄인 테스트 빌드에서 슬롯이 주기적으로 호출되는 것이 로그로 보인다.
- `MODULE_STATE_INACTIVE` 일 때 정책대로 측정·상향이 막히거나 NO-OP인지 확인한다.

7. LoRA로 Power DATA 프로토콜 검증

목적

6단계의 측정 결과를 3단계 포맷으로 묶어 LoRA 경로로 송신하고, 수신측에서 session의 `DATA` + `module_type: power` + `data_type: battery mv` (및 본문 필드)로 해석 가능함을 증명한다. session 총정리본 §4.§4.1.

산출물

- Power 측정 경로에서 `TxPacket` 조립 → `lora_module_enqueue_tx(LORA_PL_DATA, ...)` 호출
- 수신측: PC 도구·타워·또는 loopback 펌이 디코드한 필드 덤프

테스트(완료 조건)

- 한 사이클 이상: 실제 배터리 값(또는 주입 값)이 디코드 결과와 일치한다.
- `packet_id · need_ack` 정책에 따라 ACK가 오면 재전송이 멈추고, 오지 않으면 문서 정책대로 재시도한다 (로그로 확인).
- 3단계 fixture에 `battery mv` 용 케이스를 추가해 회귀 테스트에 넣는다.

일일 진행 체크리스트(복사용)

- ☐ 오늘 목표 단계 번호:
- ☐ 해당 단계 완료 조건을 위 표와 일치시켜 “통과” 체크했는가:
- ☐ 브랜치·태그·커밋 해시:
- ☐ 로그·스크린샷 저장 위치:
- ☐ 정본 문서와 달라진 점(있다면) 메모:

구현 요약 — 모듈·동작 우선

필드·예외·수치는 26-04-11 리비타 링크 펌웨어 정리.md, 26-04-11 session 기반 동작 총정리본.md 가 정보이다. 여기서는 코드를 짤 때 필요한 "누가 무엇을 하고, 무엇에 반응하는가"만 고정한다.

전체 골격(3줄)

1. Device Manager: 기기 DeviceState, 딥슬립·탈출, 각 모듈 xxx_module_set_state(ACTIVE|INACTIVE), "다음 CRON 시각·남은 ms" 계산 유틸만 맡는다. k_timer 를 실제로 거는 주체는 각 모듈이다.
2. 각 모듈: 전용 스레드 + 전용 k_msgq. ISR·라디오 IRQ·k_timer 콜백은 긴 처리를 하지 않고 큐에만 넣는다 (Zephyr 대응: 펌웨어 정리 §2.0).
3. 무선: LoRA 모듈만 라디오·TX 순서·ACK·재전송을 맡는다. 다른 모듈이 서버로 보낼 때는 lora_module_enqueue_tx(...) 한 경로로만 넘긴다.

모듈별 — 맡는 일·입력·출력(구현 기준표)

현 단계 우선순위는 펌웨어 정리 §0 (DM·Power·LoRA·RS485 우선, Valve·BLE·Security·Button 스레드 등은 스텝 가능).

모듈	맡는 것	큐·스레드	무엇에 반응하는가	무엇을 하는가
Device Manager	생명주기, 모듈 ON/OFF, CRON 유틸, 딥슬립	(설계에 따름)	버튼 GPIO(딥슬립 탈출은 Button 모듈이 아니라 DM, §3.6·§3.7), 링크 성공/실패	transition_device_state, xxx_module_set_state 호출 순서
Power	ADC·배터리, 12V 정책	power_queue + 스레드	배터리 CRON용 k_timer 만료 → 큐	슬롯에서 측정 → (전이 시) NOTIFY → 슬롯마다 DATA (battery mv) 조립 → lora_module_enqueue_tx. ACTIVE여도 12V 자동 ON 아님(request / release, §5.8)
LoRA	SX1262 등	lora_rx_queue, lora_tx_queue + 스레드	DIO1 RX IRQ; TX는 TX 큐 소비	RX: 디코드·유효 시 ack, 나머지는 module_type 별 타 모듈 큐. TX: need_ack·재전송(§6.8). ACTIVE만으로 스캔 안 함 — DM이 lora_module_start_scan() 등 별도 호출(§3.5.2)
RS485	Modbus	rs485_queue + 스레드	수집용 타이머 + CRON 유틸	다음 슬롯에만 버스 읽기 → sensor DATA → lora_module_enqueue_tx. 서버 CREATE 없는 슬롯은 상향 없음
BLE	광고 등	ble_queue	스택 콜백	ACTIVATING 부터 ACTIVE(§3.4); 현 단계 스텝 가능
Button	LED·패턴(운용)	button_queue	GPIO ISR	딥슬립 깨우기는 DM GPIO; ACTIVATED 이후는 §0 에 따라 생략 가능
Valve	밸브·유량	채널별 valve_queue	타이머·ISR·하향	현 단계 스텝
Security	틸트·부저	security_queue	GPIO·타이머	현 단계 스텝

상향 한 줄: CRON·마스크 → Power 타이머 만료 → power_queue → 스레드에서 ADC·인코드 → lora_module_enqueue_tx → lora_tx_queue → RF. 하향: RF → lora_rx_queue → 디코드·ACK → (이후)

Valve-RS485 등으로 라우팅.

Device Manager — 상태와 모듈 켜는 순서

DeviceState	한 줄
DEACTIVATED	딤슬립. 일상 RTOS 모듈 루프와 다름(§3.6).
ACTIVATING	링크 잡는 중. LoRA·BLE만 ACTIVE , Power는 INACTIVE → 배터리 주기 상향 없음.
ACTIVATED_NORMAL	현장 운용. 나머지 모듈 ACTIVE . 전환 전 RTC 동기화 등 §3.5.1.1 선행.

ACTIVATED_NORMAL 직후 권장 순서: Power → Button → RS485(즉시 수집 금지, 다음 CRON까지 타이머만) → Valve → Security(§3.5.1). Power·RS485는 ACTIVE 직후 임의 DATA 금지, 슬롯 만료 경로만(§3.5.2).

MODULE_STATE_ACTIVE 와 서버 세션(예: RS485 슬롯 CREATE)은 별개다. RS485는 활성 슬롯 있을 때만 의미 있는 DATA ; Power는 CRON 슬롯에서만 battery mv 계약(§2.7.1).

모듈 공통 — 큐 소통 방식

비동기 경로는 모듈마다 k_msgq 하나(또는 Valve처럼 채널별로 여러 개)로 통일한다. 펌웨어 정리 §2.3 이 정본이다.

- 스레드 쪽: k_msgq_get(..., K_FOREVER) 로 블로킹 대기 → 꺼낸 항목이 ModuleCmd (콜백+ param) 이거나, 모듈 전용 명령 구조체로 디스패치한다.
- 송신 쪽: DM·LoRA RX·다른 모듈이 “일 시키려면” 목적 모듈 큐에만 k_msgq_put 한다. 일반 스레드는 필요 시 K_FOREVER /타임아웃으로 블로킹해도 되고, ISR·k_timer 콜백 안에서는 반드시 k_msgq_put(..., K_NO_WAIT) 만 쓴다(블로킹 금지).
- 실행 전: module_run_state != MODULE_STATE_ACTIVE 이면 큐에서 꺼냈어도 본 처리는 하지 않는다 — 프로젝트 전체에서 같은 규칙으로 통일(§2.7).
- 동기 조회만 필요할 때: 짧은 getter(예: power_module_get_battery_mv()) 는 k_mutex 로 공유 데이터 보호. 가능하면 “일감은 큐, 읽기만 직접 호출”로 역할을 나눈다(§2.3 · §2.6).

K_NO_WAIT 실패 시: 주기 타이머 쪽은 그 주기 한 번 스킵·카운터; 하향 라우팅 쪽은 ACK 우선 송신 후 명령 폐기 가능 + lora PROGRESS module cmd enqueue failed 로 진단 상향 권장(§2.3.1 · §2.3.2 , session §1.1.6.2).

모듈 공통 — 공용 함수(모듈 경계에서만 쓰는 진입점)

여러 모듈이 같은 페이로드 조립을 복붙하지 않도록, 아래는 “한 군데만 구현”하는 쪽이 문서와 맞다.

- lora_module_enqueue_tx(LoraPayloadType, TxPacket*) — 서버로 나가는 모든 무선 상향의 공통 출구. Power·RS485 등은 라디오를 직접 건드리지 않고 여기까지만 호출(§2.5 , §6.8).
- 큐 적재 실패·진단 상향 조립은 module_fail_report_emit(...) 같은 단일 함수로 모아 lora_module_enqueue_tx(LORA_PL_PROGRESS, ...) 호출(§2.3.2).
- 인코드·디코드(session 바이트 레이아웃)은 LoRA 모듈 안에만 두거나, protocol/ 에 두고 LoRA·테스트가 공유 — 어느 쪽이든 “한 표”와 싱크 유지(§6.4).

Flash·레지스터 맵 등 진짜 공유 자원은 k_mutex 로 보호한다(§2.6).

Session·이 로드맵에서 쓰는 페이로드

- 프레임: 비암호 node_id → 암호 구간 packet_id , type , module_type + 본문(ACK 는 본문 없음). 바이트 표는 session §8 ·펌웨어 §6.4 와 한 장으로 맞출 것.
- LORA_PL_DATA 등 = session DATA 등 1:1(§8.8).

용도	module_type	type	본문 요지
수신 확인	동일	ACK	없음
배터리 mV	power	DATA	data_type battery mv , battery_mv
저전압·복구	power	NOTIFY	알림 종류만
큐 실패	lora	PROGRESS	module cmd enqueue failed + target_module_type , reason_code 등

하향은 장기적으로 CREATE / DELETE / UPDATE 만 쓰고 기존 CMD enum은 수신에서 흡수(session §8.9). 나머지 타입·필드는 정본 절을 연다.

참고: 로드맵 순서(LoRA 먼저, DM 생명주기는 그다음)

- 직렬화(3) 후 LoRA(4)로 두면 RF 버그와 바이트 버그를 나눌 수 있다.
- DM 생명주기(5)는 LoRA가 돈 뒤 없으면 회귀 범위가 명확하다.

세부는 펌웨어 정리 §0 ~ §8 , session §1 · §8 을 본다.